

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

AIM

To implement and simulate register transfer operations and basic arithmetic and logical operations such as addition, subtraction, AND, and OR using the 8085 microprocessor, and to study how data is transferred between registers and processed by the ALU.

APPARATUS REQUIRED

1. Computer/Laptop
2. GNUSim8085 Simulator
3. 8085 instruction set
4. 8085 registers and ALU
5. Two 8-bit input data values

PROCEDURE

1. Enter two 8-bit data values into the specified memory locations.
2. Load the first data value into register B and the second value into register C.
3. Transfer the required data from the registers to the accumulator.
4. Use the 8085 ALU to perform addition, subtraction, AND, and OR operations.

5. Store the results in suitable registers or memory locations.
6. Run the program in GNUSim8085 and observe the register contents and results.
7. Record the obtained results and verify them with the expected values.

PROGRAM

The screenshot displays the GNUSim8085 simulator interface. The main window shows the assembly program being executed. The registers window on the left shows the current state of the 8085 registers. The memory window on the right shows the memory locations and their contents.

Registers:

Register	Value
A	35
BC	25 15
DE	08 02
HL	01 07
PSW	00 00
PC	42 1D
SP	FF FF
Int-Reg	00

Flags:

Flag	Value
S	0
Z	0
AC	0
P	1
C	0

Assembly Program:

```

1 LDA 2050H
2 MOV B,A
3
4 LDA 2051H
5 MOV C,A
6
7 ; ADDITION
8 MOV A,B
9 ADD C
10 STA 2052H
11
12 ; SUBTRACTION
13 MOV A,B
14 SUB C
15 STA 2053H
16
17 ; AND
18 MOV A,B
19 ANA C
20 STA 2054H
21
22 ; OR
23 MOV A,B
24 ORA C
25 STA 2055H
26
27 HLT
  
```

Memory:

Address (Hex)	Address	Data
2050	8272	37
2051	8273	21
2052	8274	58
2053	8275	16
2054	8276	5
2055	8277	53
2056	8278	0
2057	8279	0
2058	8280	0
2059	8281	0
205A	8282	0
205B	8283	0
205C	8284	0
205D	8285	0

Assembler Message:

```

Line No  Assembler Message
0         Program assembled successfully
  
```

RESULT

The register transfer and ALU operations were successfully simulated. The data was transferred between registers and processed by the ALU to perform addition, subtraction, AND, and OR operations successfully.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

AIM

To simulate data transfer operations between registers and memory and study the difference between single-bus and multi-bus organization in terms of data transfer efficiency and execution time.

APPARATUS REQUIRED

1. Computer/Laptop
2. GNUSim8085 simulator
3. 8085 processor simulator
4. Memory locations

5. Registers and buses

PROCEDURE

1. Store input data in different memory locations.
2. Transfer the data from memory to registers using the accumulator.
3. Transfer data between different registers.
4. Store the processed data back into memory.
5. Compare sequential single-bus transfers with parallel multi-bus transfers and analyze the reduction in transfer time.

PROGRAM

The screenshot displays a microprocessor simulator interface. On the left, the 'Registers' panel shows the state of various registers: A (6C), BC (36 24), DE (36 02), HL (01 07), PSW (00 00), PC (42 15), SP (FF FF), and Int-Reg (00). The 'Flag' panel shows S (0), Z (0), AC (0), P (1), and C (0). Below these is a 'Decimal - Hex Conversion' section with input fields for decimal and hex values, both currently set to 0. The 'I/O Ports' section shows a value of 0. The 'Memory' section also shows a value of 0. The central assembly window lists 18 instructions: 1 LDA 2100H, 2 MOV B,A, 3, 4 LDA 2101H, 5 MOV C,A, 6, 7 LDA 2102H, 8 MOV D,A, 9, 10 MOV A,B, 11 ADD C, 12 MOV B,A, 13, 14 MOV A,B, 15 ADD D, 16 STA 2103H, 17, 18 HLT. On the right, the 'Memory' tab is active, showing a table of memory addresses and data. The 'Start' address is 2100h. The table lists addresses from 2100 to 210D in hex, with corresponding addresses in decimal (8448 to 8461) and data values (18, 36, 54, 108, and then zeros). At the bottom, the 'Assembler Message' pane shows '0 Program assembled successfully'.

Address (Hex)	Address	Data
2100	8448	18
2101	8449	36
2102	8450	54
2103	8451	108
2104	8452	0
2105	8453	0
2106	8454	0
2107	8455	0
2108	8456	0
2109	8457	0
210A	8458	0
210B	8459	0
210C	8460	0
210D	8461	0

RESULT

The data transfer operations between memory and registers were successfully simulated. The multi-bus organization provides faster and more parallel data transfers than the single-bus organization, thereby reducing bus bottlenecks and improving system efficiency

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

AIM

To simulate a five-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory, and Write-back stages and compare its execution time, speedup, and throughput with a non-pipelined system.

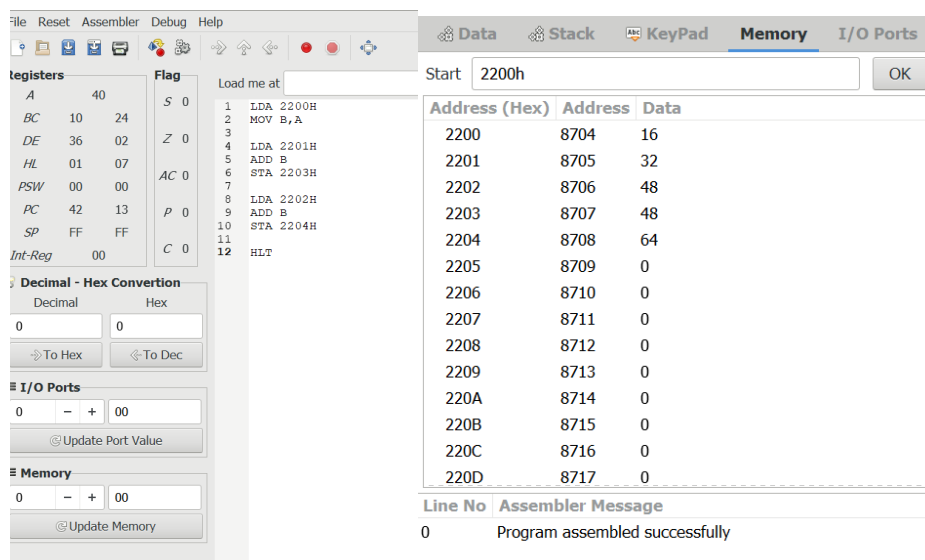
APPARATUS REQUIRED

1. Computer/Laptop
2. GNUSim8085 simulator
3. 8085 instruction set
4. Pipeline stage model
5. Timing/cycle analysis

PROCEDURE

1. Define the five pipeline stages: Fetch, Decode, Execute, Memory, and Write-back.
2. Load a sequence of instructions into the processor.
3. Execute the instructions sequentially to represent a non-pipelined processor.
4. Execute the same instructions using overlapping pipeline stages.
5. Calculate execution time, throughput, and speedup and compare both methods.

PROGRAM



RESULT

The five-stage instruction pipeline was successfully simulated conceptually. Pipeline execution improves instruction throughput by overlapping instruction stages. Compared with non-pipelined execution, pipelining significantly reduces the total execution time for multiple instructions.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

AIM

To study data, control, and structural hazards in a pipelined processor and analyze hazard-resolution techniques such as stalling, data forwarding, and branch prediction.

APPARATUS REQUIRED

1. Computer/Laptop
2. GNUSim8085 simulator
3. 8085 instruction set
4. Pipeline stage model
5. Timing diagram/hazard analysis

PROCEDURE

1. Execute a sequence of dependent instructions to generate a data hazard.
2. Introduce a branch instruction to generate a control hazard.
3. Use common resources simultaneously to demonstrate a structural hazard.
4. Apply stalling and forwarding to resolve data and structural hazards.
5. Apply branch prediction to reduce the effect of control hazards and compare the execution time.

PROGRAM

The screenshot displays the GNUSim8085 Microprocessor Simulator interface. The top menu bar includes File, Reset, Assembler, Debug, and Help. Below the menu is a toolbar with icons for file operations and simulation control. The main window is divided into several sections:

- Registers:** A table showing the status of 8085 registers. The PC (Program Counter) is at 42 1A, and the SP (Stack Pointer) is at FF FF.
- Flags:** A section showing the status of the S, Z, AC, P, and C flags.
- Decimal - Hex Conversion:** A section with input fields for decimal and hex values and buttons for conversion.
- I/O Ports:** A section with input fields for port values and buttons for updating.
- Memory:** A section showing a memory dump starting at address 2300h. The dump shows a sequence of data values stored in memory.
- Assembler Message:** A section at the bottom showing the message "Program assembled successfully".

The memory dump table is as follows:

Address (Hex)	Address	Data
2300	8960	16
2301	8961	5
2302	8962	21
2303	8963	17
2304	8964	0
2305	8965	0
2306	8966	0
2307	8967	0
2308	8968	0
2309	8969	0
230A	8970	0
230B	8971	0
230C	8972	0
230D	8973	0

RESULT:

Thus, data and control hazards were successfully demonstrated using 8085 instructions. Stalling and other hazard-handling techniques help to avoid incorrect execution and improve pipeline performance.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

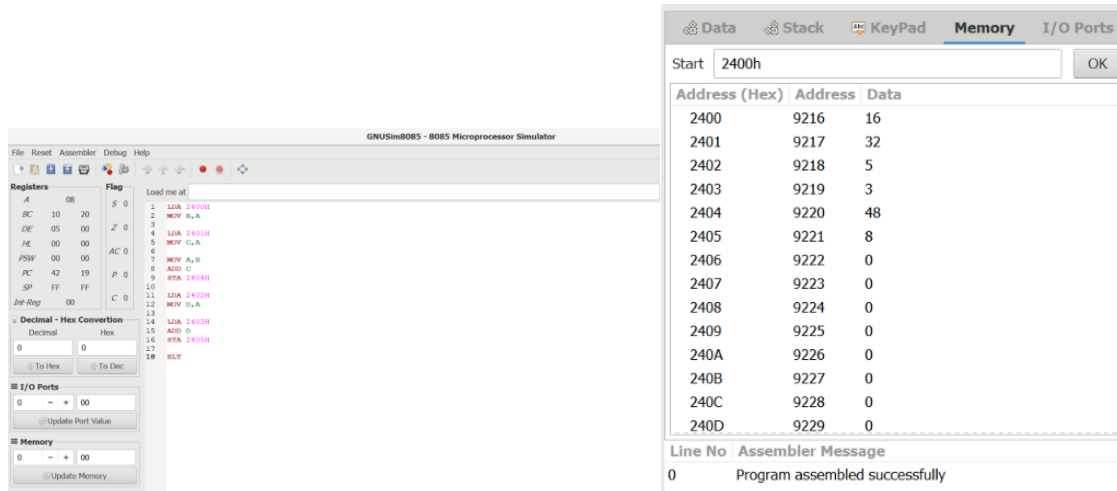
AIM:

To demonstrate the concepts of superscalar execution, out-of-order execution, speculative execution, hyper-threading, and simultaneous multithreading (SMT) using an 8085-based conceptual simulation.

ALGORITHM:

1. Create two independent instruction streams.
2. Load different data for each instruction stream.
3. Execute arithmetic operations for the first stream.
4. Execute arithmetic operations for the second stream.
5. Store the results in separate memory locations.
6. Represent the two streams as separate threads.
7. Compare sequential execution with the concept of parallel execution.
8. Analyze instruction throughput and resource utilization.

PROGRAM



The screenshot displays the GNUSim8085 - 8085 Microprocessor Simulator interface. The top menu bar includes File, Reset, Assembler, Debug, and Help. The main window is divided into several sections:

- Registers:** Shows the status of various registers including A, BC, DE, HL, PSW, PC, SP, and Int-Flag.
- Flag:** Displays the status of the Zero (Z), Carry (C), and Overflow (O) flags.
- Assembly Code:** A list of instructions being executed, such as LDA, MOV, ADD, and RST.
- Memory:** A table showing the memory dump starting from address 2400h. The table has three columns: Address (Hex), Address, and Data.
- I/O Ports:** A section for monitoring and updating I/O port values.
- Assembler Message:** A log showing the status of the assembly process, indicating "Program assembled successfully".

Address (Hex)	Address	Data
2400	9216	16
2401	9217	32
2402	9218	5
2403	9219	3
2404	9220	48
2405	9221	8
2406	9222	0
2407	9223	0
2408	9224	0
2409	9225	0
240A	9226	0
240B	9227	0
240C	9228	0
240D	9229	0

RESULT:

Thus, the concepts of parallel instruction execution and multithreading were successfully demonstrated conceptually. The 8085 executes instructions sequentially, while modern superscalar processors can execute multiple independent instructions in parallel

Code of Conduct:

I K.AISHWARYA (192572116)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation